

Relatório: TAD GRAPH e a determinação de conjuntos de vértices dominantes

Universidade de Aveiro

Dept. de Eletrónica, Telecomunicações e Informática

Gabriel Guzmán (126180) , Henrique Tardelli (125087)

23 de dezembro de 2025

1 Apresentação e Aspetos gerais

Este trabalho teve como principal objetivo o aprofundamento do estudo de algoritmos e estruturas de dados aplicados a grafos, recorrendo à utilização do TAD GRAPH e de TAD's auxiliares, como o INDICES-SET. A abordagem adotada permitiu consolidar conceitos fundamentais como representação de grafos através de listas de adjacência, manipulação de conjuntos de vértices e desenvolvimento de algoritmos sobre estruturas abstratas de dados.

Ao longo do trabalho, procedeu-se à análise das estruturas fornecidas, à implementação de funções previamente incompletas e ao desenvolvimento de algoritmos para a determinação de conjuntos de vértices dominantes em grafos não orientados.

Os algoritmos implementados foram testados com grafos simples e mais complexos, garantindo a sua correção funcional. Para além disso, foi realizada uma análise da complexidade computacional dos algoritmos de procura exaustiva usados para determinar conjuntos dominantes mínimos.

2 Análise da função GraphComputeMinDominatingSet()

A função **GraphComputeMinDominatingSet()** tem como objetivo encontrar o conjunto dominante de menor cardinalidade (menor número de vértices). A estratégia implementada baseia-se na procura exaustiva, que consiste em gerar e testar todos os subconjuntos possíveis de vértices do grafo.

A métrica principal adotada para a análise da complexidade é o número de iterações do ciclo principal em função do número de vértices V do grafo (denotado por n). Adicionalmente, considera-se o custo da verificação de dominância feita pela função auxiliar **GraphIsDominatingSet()**.

O espaço de procura é o conjunto das partes do conjunto de vértices, o que implica que existem 2^n subconjuntos possíveis.

2.1 Complexidade Teórica

O algoritmo percorre um ciclo (do...while) controlado pela função **IndicesSetNextSubset**, que gera sequencialmente todas as 2^n combinações binárias de vértices. Dentro do ciclo, realiza-se a validação do conjunto, cuja complexidade é proporcional ao número de vértices e arestas, aproximadamente $O(n + E)$.

Portanto, a complexidade global é dada por:

$$O(2^n \cdot n)$$

Esta é uma complexidade exponencial.

2.2 Melhor caso

No contexto deste algoritmo de procura exaustiva, o termo "melhor caso"refere-se apenas ao trabalho realizado dentro de cada iteração.

- O algoritmo foi feito de modo a não verificar a dominância (**GraphIsDominatingSet()**) se o tamanho do subconjunto candidato for igual ou superior ao melhor tamanho já encontrado (minSize).
- O melhor cenário ocorre quando encontramos um conjunto dominante muito pequeno (por exemplo, de tamanho 1 ou 2) logo no início da geração dos subconjuntos. Isto

ativa a otimização logo no início, evitando a chamada da função de verificação (que é custosa) para a grande maioria dos 2^n subconjuntos restantes.

Contudo, mesmo no melhor caso, o algoritmo não termina antecipadamente. Ele é obrigado a gerar todos os 2^n subconjuntos para garantir que não existe nenhum outro conjunto válido com cardinalidade inferior. Assim, o tempo continua a ser exponencial 2^n.

2.3 Pior caso

O pior caso ocorre quando as otimizações implementadas têm pouco ou nenhum efeito.

- Isto acontece se o conjunto dominante mínimo tiver uma cardinalidade elevada (próxima de n).
- Neste cenário, a condição de paragem da otimização (**currentSize** $<$ **minSize**) raramente é satisfeita, obrigando o algoritmo a executar a função **GraphIsDominatingSet** para uma grande percentagem dos 2^n subconjuntos.

Para um grafo com $n = 20$, por exemplo, o algoritmo realiza $2^{20} \approx 1.000.000$ iterações, crescendo de forma abrupta conforme aumentamos n. O pior caso confirma a natureza difícil de se lidar do problema para grafos grandes utilizando esta abordagem exata.

2.4 Caso médio e Conclusão da Análise

O caso médio não difere significativamente do pior caso em termos de ordem de grandeza. A natureza combinatória do problema impõe que, independentemente da estrutura do grafo, o número de subconjuntos a analisar cresça exponencialmente.

Graficamente, a curva de tempo de execução em função de n apresenta um crescimento extremamente acentuado (curva exponencial). A solução é eficiente para instâncias pequenas, mas torna-se inviável para instâncias maiores, onde seriam necessárias heurísticas ou algoritmos de aproximação em vez de procura exaustiva.

```
=== ANALISE DE COMPLEXIDADE: GraphComputeMinDominatingSet ===
| Vertices   | Iteracoes       | Tempo (s)       |
|------------|-----------------|-----------------|
| 2          | 4               | 0.000005        |
| 4          | 16              | 0.000008        |
| 6          | 64              | 0.000016        |
| 8          | 256             | 0.000081        |
| 10         | 1024            | 0.000261        |
| 12         | 4096            | 0.000869        |
| 14         | 16384           | 0.003894        |
| 16         | 65536           | 0.011277        |
| 18         | 262144          | 0.030805        |
| 20         | 1048576         | 0.109512        |
| 22         | 4194304         | 0.472303        |
|------------|-----------------|-----------------|
```

Figura 1: Resultados experimentais: Número de iterações e tempo de execução em função do número de vértices (N).

3 Análise da função GraphComputeMinWeightDominatingSet()

A função **GraphComputeMinWeightDominatingSet()** tem como objetivo encontrar o conjunto dominante com o menor peso total (soma dos pesos dos vértices constituintes). A estratégia implementada segue a mesma abordagem de procura exaustiva utilizada na função anterior, gerando e testando sequencialmente todos os subconjuntos possíveis de vértices.

Para a análise da complexidade, consideram-se as variáveis N (número de vértices) e M (número de arestas). O espaço de procura mantém-se inalterado em 2^N subconjuntos.

3.1 Complexidade Teórica

O algoritmo utiliza um ciclo que percorre todos os 2^N subconjuntos. Em cada iteração, existe um custo fixo de cálculo do peso ($O(N)$) e um custo condicional de verificação de dominância ($O(N+M)$).

Portanto, a complexidade assimptótica global no pior cenário é dada por:

$$O((N+M)\cdot 2^N)$$

Esta complexidade mantém a natureza exponencial característica da procura exaustiva.

3.2 Melhor caso

O termo "melhor caso"refere-se ao cenário onde o mecanismo é o mais eficaz possível dentro das iterações.

- Isto acontece se o algoritmo encontrar uma solução ótima (um conjunto dominante com peso muito baixo) logo nas primeiras iterações do ciclo.
- Neste cenário, a condição de paragem da otimização (currentTotalWeight >= minTotalWeight) é avaliada como verdadeira para a vasta maioria dos subconjuntos.
- Consequentemente, a função custosa **GraphIsDominatingSet** não é executada. O custo reduz-se ao cálculo do peso, que envolve percorrer o array de índices ($O(N)$).

O cálculo da complexidade para o melhor caso é:

$$T_{melhor}(N) \approx O(N\cdot 2^N)$$

3.3 Pior caso

O pior caso verifica-se quando o mecanismo de poda é ineficaz.

- Isto ocorre se os pesos dos vértices forem distribuídos de forma a que o algoritmo encontre sucessivamente soluções apenas ligeiramente melhores no final da execução, ou se a condição de peso for sempre satisfeita (ex: pesos nulos).
- A condição de poda falha frequentemente, obrigando à execução da verificação de dominação.
- A função **GraphIsDominatingSet** tem complexidade $O(N+M)$, pois percorre vértices e arestas adjacentes.

O cálculo da complexidade para o pior caso é a soma do custo do cálculo do peso com o custo da verificação de dominância, repetido para todo o espaço de procura:

$$T_{pior}(N,M) \approx O((N+M)\cdot 2^N)$$

3.4 Caso médio e Conclusão da Análise

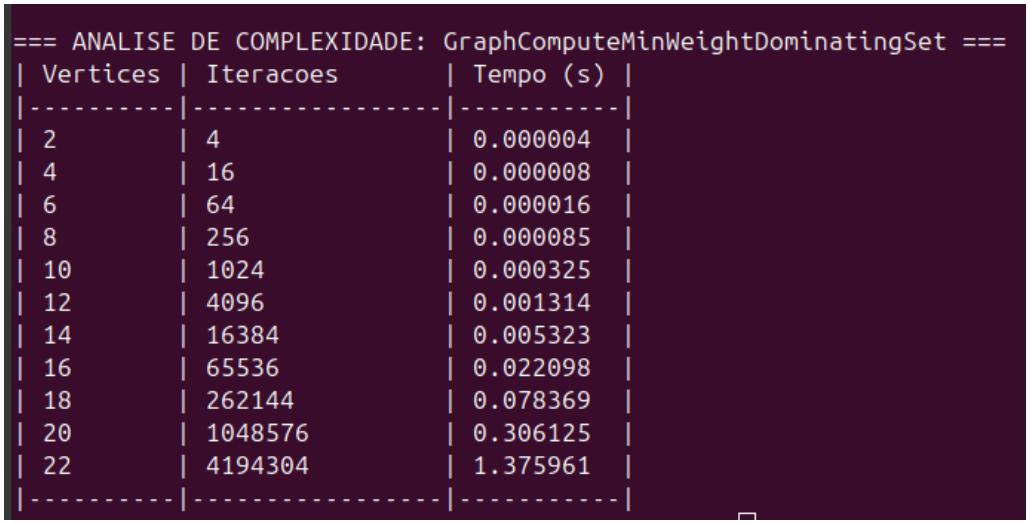
No caso médio, assume-se uma distribuição aleatória dos pesos e da topologia do grafo. Embora a poda ocorra em algumas iterações, a estrutura fundamental do algoritmo obriga à geração de todos os subconjuntos para garantir a otimalidade global.

A probabilidade de entrar no bloco de verificação não elimina o fator exponencial 2^N. Matematicamente, o custo situa-se entre o melhor e o pior caso, mas em análise de algoritmos exponenciais, o termo 2^N é o fator dominante.

Assim, a complexidade do caso médio é limitada superiormente pelo pior caso:

$$T_{medio}(N, M) \approx O((N + M) \cdot 2^N)$$

Conclui-se que o algoritmo, tal como o anterior, é impraticável para valores elevados de N (intratável), crescendo exponencialmente com o número de vértices.



```
=== ANALISE DE COMPLEXIDADE: GraphComputeMinWeightDominatingSet ===
| Vertices | Iteracoes       | Tempo (s) |
|----------|-----------------|-----------|
| 2        | 4               | 0.000004  |
| 4        | 16              | 0.000008  |
| 6        | 64              | 0.000016  |
| 8        | 256             | 0.000085  |
| 10       | 1024            | 0.000325  |
| 12       | 4096            | 0.001314  |
| 14       | 16384           | 0.005323  |
| 16       | 65536           | 0.022098  |
| 18       | 262144          | 0.078369  |
| 20       | 1048576         | 0.306125  |
| 22       | 4194304         | 1.375961  |
|----------|-----------------|-----------|
```

Figura 2: Resultados experimentais: Número de iterações e tempo de execução em função do número de vértices (N).

4 Conclusão

A realização deste trabalho permitiu atingir com sucesso os objetivos propostos, proporcionando uma compreensão mais aprofundada da manipulação de grafos e da implementação de algoritmos sobre estas estruturas. A determinação de conjuntos de vértices dominantes revelou-se relevante, não só do ponto de vista teórico, mas também pela sua aplicabilidade em problemas reais.

Os resultados obtidos confirmam que os algoritmos de procura exaustiva, embora corretos e simples de implementar, apresentam uma complexidade elevada $O(2^n \cdot C_1)$, tornando-se rapidamente impraticáveis para grafos de maior dimensão. Esta constatação evidencia a importância do estudo de algoritmos mais eficientes para resolver problemas deste tipo.

Em suma, o trabalho constituiu uma experiência formativa valiosa, reforçando competências ao nível da programação em C, da utilização de tipos abstratos de dados, da análise de complexidade e da validação experimental de algoritmos.